Garbage Recognizer: A Deep Neural Network for Waste Image Classification

Salman Askarov
Areg Karapetyan
Master in Data Science for Economics
Università degli Studi di Milano
Algorithms for Massive Data — A.Y. 2025/26

Abstract

This report describes the design, implementation and evaluation of a Deep Neural Network (DNN) for the classification of waste images into ten material categories. We use the Garbage Classification v2 dataset and implement a custom convolutional neural network trained from scratch architecture. We perform model selection over the learning rate and dropout rate, and evaluate the generalisation capability of the resulting model on a held-out test set. The final model achieves a test accuracy of **67.0%** across the ten classes. We discuss the results, the per-class performance, the absence of overfitting, and the scalability of the proposed pipeline.

1 Introduction

Automated waste sorting is an increasingly important application of computer vision, with direct relevance to recycling efficiency and environmental sustainability. The task addressed in this project is to classify images of discarded items into their correct material category, so that — in a real-world deployment such as a recycling facility conveyor belt — each item could be routed automatically to the appropriate processing stream.

We approach this as a supervised image classification problem and implement a custom Deep Neural Network using the PyTorch framework. In accordance with the project requirements, we perform model selection over a set of hyperparameters and evaluate the generalisation capability of the trained model. The complete implementation is provided as a single Jupyter notebook executable on Google Colab.

2 Dataset

2.1 Source and license

We use the Garbage Classification v2 dataset, published on Kaggle under the MIT license. The dataset is downloaded programmatically at runtime through the Kaggle API, authenticating with credentials supplied as environment variables. In the submitted version of the notebook these credentials are replaced with the placeholder `"xxxxxx"` so as not to expose sensitive information.

2.2 Structure and classes considered

The dataset is organised into folders, where each folder name corresponds to a class label. The images are grouped under an `original` directory and span the following **ten classes**: battery,

biological, cardboard, clothes, glass, metal, paper, plastic, shoes and trash. We consider all ten classes. The dataset also provides pre-resized variants (`standardized_256`, `standardized_384`); we deliberately use the `original` images so that resizing is handled consistently within our own preprocessing pipeline.

In total the dataset contains **12,258 images**. The class distribution, shown in Figure 1, is noticeably imbalanced: the largest class (clothes, 1,892 images) contains roughly four times as many samples as the smallest (trash, 452 images). This imbalance is accounted for in the data splitting strategy and is reflected in the per-class results.

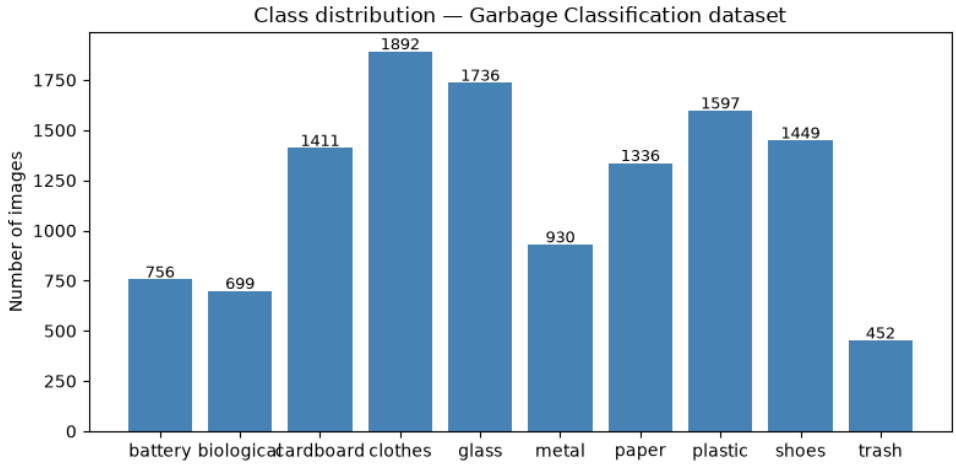

Figure 1: Distribution of images across the ten classes. The dataset is imbalanced, ranging from 452 (trash) to 1,892 (clothes) images per class.

2.3 Subsampling

Following the project guidelines, the notebook exposes a global boolean variable `SUBSAMPLE` which, when enabled, restricts the dataset to a fixed number of images per class. This was used during development to iterate quickly. All final results reported here were obtained with `SUBSAMPLE = False`, i.e. on the full dataset.

3 Data Organisation and Preprocessing

3.1 Train / validation / test split

The data are split into three disjoint subsets: 80% for training, 10% for validation and 10% for testing. The split is *stratified* by class so that the relative frequency of each category is preserved across all three subsets, which is important given the class imbalance described above. A fixed random seed is used throughout to ensure reproducibility. The resulting test set contains 1,226 images.

3.2 Image transforms

All images are resized to a fixed resolution before being passed to the network. Pixel values are converted to tensors and normalised using the standard ImageNet channel means and standard deviations, which centres the input distribution and stabilises training.

3.3 Data augmentation

To improve generalisation and reduce overfitting, the training set is augmented on-the-fly with random transformations: horizontal and vertical flips, small rotations, and colour jitter (bright-

ness, contrast, saturation and hue). These transformations preserve the semantic class of each image — a plastic bottle remains plastic whether flipped or slightly rotated — while exposing the model to greater visual variety. Crucially, augmentation is applied *only* to the training set; the validation and test sets use a deterministic transform (resize, tensor conversion, normalisation) so that evaluation is performed on clean, consistent images.

4 Model and Algorithms

4.1 Architecture

We implement a custom convolutional neural network from scratch, following a **VGG-style** design. The network consists of four convolutional blocks with progressively increasing channel depth (32, 64, 128, 256), each block containing two 3×3 convolutional layers followed by batch normalisation, ReLU activation, max pooling and spatial dropout. The convolutional feature extractor is followed by an adaptive average pooling layer and a three-layer fully connected classifier head that maps the learned features to the ten output classes. In total the network comprises eight convolutional layers and three fully connected layers.

4.2 Justification of the design

The VGG-style architecture was chosen for several reasons. First, it is a well-established and empirically validated design pattern, providing a principled baseline rather than an arbitrary one. Second, the use of small 3×3 filters offers a favourable trade-off between receptive-field size and parameter count: two stacked 3×3 convolutions have the same effective receptive field as a single 5×5 convolution but with fewer parameters and an additional non-linearity. This is advantageous when training from scratch on a dataset of limited size. Third, the uniform, repetitive block structure made the architecture straightforward to reason about and adjust during hyperparameter selection. Rather than adopting the full sixteen-layer VGG configuration, which would risk overfitting on our dataset, we designed a shallower four-block variant that retains the core VGG principles while remaining appropriately scaled to the task.

4.3 Training procedure

The network is trained end-to-end using the Adam optimiser and the cross-entropy loss function. A step learning-rate scheduler progressively reduces the learning rate during training. The best model, as measured by validation accuracy, is retained across epochs. Because the network learns all of its features from scratch — with no pretrained weights — it requires more epochs to converge than a transfer-learning approach would.

5 Hyperparameter Selection

In line with the project requirement to perform model selection, we conducted a search over the two hyperparameters that most strongly influence a from-scratch CNN: the **learning rate** and the **dropout rate**. Three configurations were each trained for eight epochs on the same train/validation split, and the configuration achieving the highest validation accuracy was selected for the final, full-length training run. Conducting the search over short runs allows the configurations to be *ranked* reliably without incurring the cost of fully training each one. The results are summarised in Table 1 and Figure 2.

Table 1: Hyperparameter configurations evaluated during model selection. The third configuration achieved the best validation accuracy and was selected for final training.

Configuration	Learning rate	Dropout	Best val. accuracy
Config 1	1×10^{-3}	0.2	0.5082
Config 2	1×10^{-3}	0.4	0.5171
Config 3	$\mathbf{5 \times 10^{-4}}$	**0.3**	**0.5367**

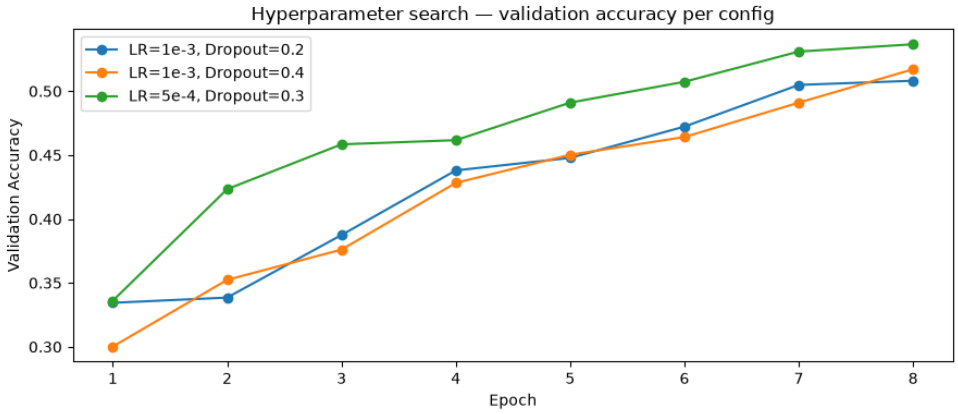

Figure 2: Validation accuracy per epoch for the three hyperparameter configurations evaluated. The configuration with learning rate 5×10^{-4} and dropout 0.3 consistently outperformed the others.

The selected configuration uses a learning rate of 5×10^{-4} and a dropout rate of 0.3. The lower learning rate evidently allowed for more stable optimisation, while the moderate dropout provided a good balance between regularisation and capacity.

6 Experiments and Results

6.1 Experimental setup

The selected configuration was used to train the final model on the full dataset for 60 epochs. Training and validation loss and accuracy were recorded at every epoch. The experiments were run both on Google Colab (T4 GPU) and on a local machine equipped with an NVIDIA RTX 3050 GPU; the notebook is device-agnostic and automatically uses a GPU when available.

6.2 Training behaviour

Figure 3 shows the training and validation curves. The model improved steadily over the first ∼40 epochs before plateauing: validation accuracy increased from roughly 30% in the first epoch to approximately 66% at convergence. Notably, the training and validation accuracies remained close throughout, with validation accuracy in fact slightly *exceeding* training accuracy. This indicates that the model was **not overfitting** — the data augmentation and dropout regularisation were effective. The convergence was confirmed by comparing runs of 40 and 60 epochs: extending training from 40 to 60 epochs improved best validation accuracy only marginally (from 65.3% to 66.2%), demonstrating that the model had reached the natural capacity ceiling of this architecture.

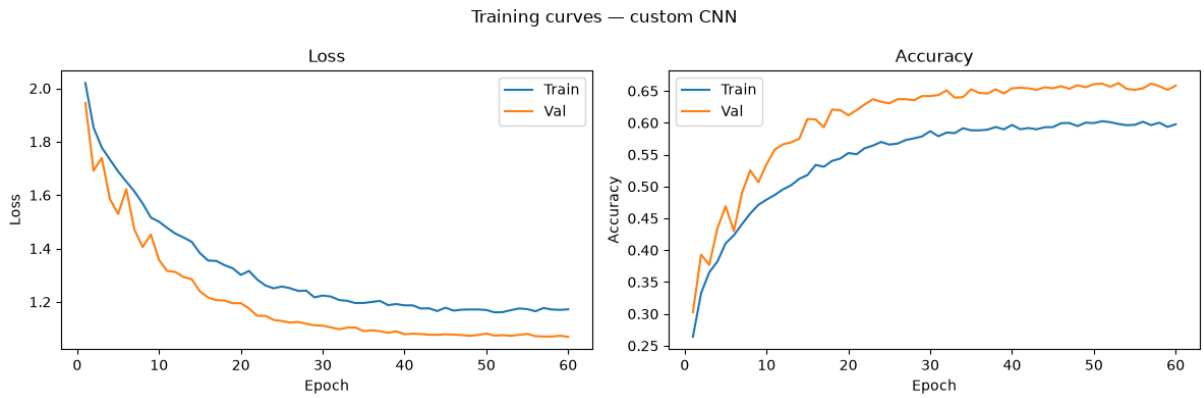

Figure 3: Training and validation loss (left) and accuracy (right) over 60 epochs. The close tracking of the two curves indicates the absence of overfitting; the plateau after ∼40 epochs indicates convergence.

6.3 Test set performance

On the held-out test set of 1,226 images, the final model achieved a **test accuracy of 67.0%** (test loss 1.029). This is consistent with the validation accuracy, confirming that the model generalises well to unseen data and did not overfit to the validation set during model selection.

Table 2: Per-class precision, recall and F1-score on the test set.

Class	**Precision**	**Recall**	**F1-score**	**Support**
battery	0.71	0.64	0.68	76
biological	0.69	0.81	0.75	70
cardboard	0.70	0.79	0.74	141
clothes	0.77	0.81	0.79	189
glass	0.62	0.78	0.69	174
metal	0.54	0.31	0.39	93
paper	0.60	0.58	0.59	134
plastic	0.66	0.61	0.64	159
shoes	0.63	0.61	0.62	145
trash	0.82	0.51	0.63	45
Accuracy		**0.67** (1226 samples)		
Macro avg	0.67	0.65	0.65	1226
Weighted avg	0.67	0.67	0.66	1226

6.4 Per-class analysis and confusion matrix

The per-class results in Table 2 and the confusion matrix in Figure 4 reveal substantial variation across categories. The best-performing classes are **clothes** (F1 = 0.79) and **biological** (F1 = 0.75), which have visually distinctive appearances. The weakest class by far is **metal** (F1 = 0.39, recall = 0.31): the confusion matrix shows that 25% of metal images are misclassified as glass, reflecting the genuine visual similarity between reflective metallic and glass surfaces. Other notable confusions include trash being mistaken for glass and plastic (trash is an inherently heterogeneous "catch-all" category), and paper occasionally being confused with cardboard. These errors are intuitive and reflect real visual ambiguities in the materials rather than arbitrary model failures.

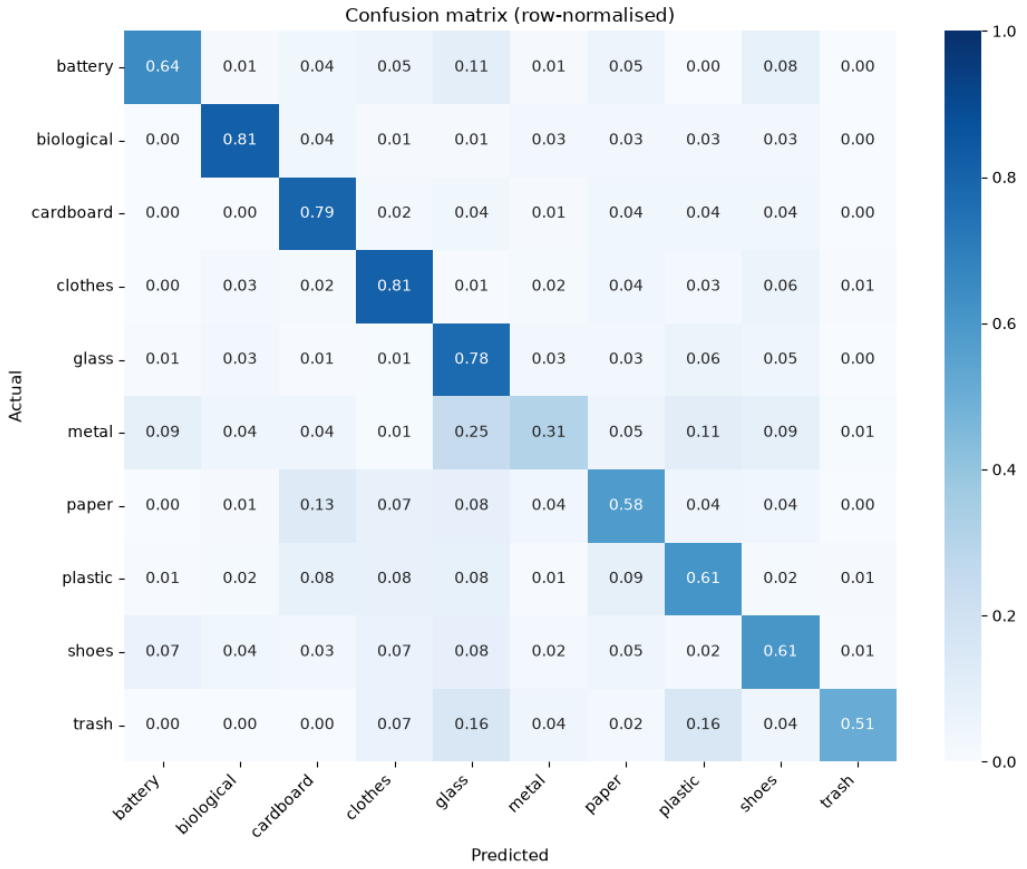

Figure 4: Row-normalised confusion matrix on the test set. Diagonal entries are correct classifications. The most prominent off-diagonal error is metal being classified as glass (0.25).

6.5 Qualitative examples

Figure 5 shows a selection of misclassified test images together with their true and predicted labels. These examples illustrate that many errors occur on genuinely ambiguous images — for instance items photographed against backgrounds that share colours or textures with another material category.

Figure 5: Examples of misclassified test images, with true and predicted labels. Many errors arise from genuine visual ambiguity between material categories.

6.6 Discussion

A test accuracy of 67.0% is a reasonable result for a convolutional network trained entirely from scratch on a ten-class problem with a moderately sized and imbalanced dataset. We deliberately chose to design and train our own network rather than fine-tuning a large pretrained model, as this demonstrates an understanding of convolutional architecture design from first principles, which was the intended learning objective. It is worth noting that transfer-learning approaches based on large networks pretrained on ImageNet (such as ResNet or EfficientNet) typically achieve higher accuracy on tasks of this kind, since they benefit from features learned on millions of images; our from-scratch network, by contrast, must learn all visual features from the comparatively limited data available here. The main avenue for improvement within our own approach would be to address the class imbalance (e.g. through a weighted loss or oversampling), which would likely raise the recall of under-represented and frequently-confused classes such as metal and trash.

7 Scalability

A central requirement of the project is that the solution scales to datasets larger than the one used. Several design decisions support this:

- **Streaming data loading.** Images are loaded in mini-batches from disk by PyTorch's `DataLoader`, using parallel worker processes. Images are never all held in memory simultaneously, so memory usage remains constant irrespective of dataset size.
- **Subsampling switch.** A single global variable (`SUBSAMPLE`) toggles between a development subset and the full dataset, with no other code changes required.
- **Compact architecture.** The custom CNN has substantially fewer parameters than large pretrained networks such as ResNet50, so each training step is computationally cheaper and the model fits comfortably on modest hardware (e.g. a 6 GB laptop GPU) even with reasonable batch sizes.
- **Batch-based processing.** Training, validation, evaluation and the hyperparameter search all operate on fixed-size batches. A larger dataset only increases the number of batches per epoch, not the per-batch memory footprint.

8 Reproducibility

All experiments are fully reproducible. Random seeds are fixed for the data split and for PyTorch operations. The dataset is obtained programmatically via the Kaggle API, and all preprocessing, training and evaluation steps are contained in a single self-contained Jupyter notebook that runs end-to-end on Google Colab. A badge linking directly to the Colab version of the notebook is provided in the GitHub repository.

9 Conclusion

We have implemented and evaluated a custom Deep Neural Network for the classification of waste images into ten categories. Following a VGG-style design, the network was trained from scratch, with model selection performed over the learning rate and dropout rate, and its generalisation capability assessed on a held-out test set. The final model reached a test accuracy of 67.0%, with strong performance on visually distinctive classes such as clothes and biological waste, and weaker performance on visually ambiguous classes such as metal. We discussed the model's relationship to transfer-learning approaches, the absence of overfitting, the convergence behaviour of the model, and the scalability of the pipeline to larger datasets.

Declaration

We declare that this material, which we now submit for assessment, is entirely our own work and has not been taken from the work of others, save and to the extent that such work has been cited and acknowledged within the text of our work, and including any code produced using generative AI systems. We understand that plagiarism, collusion, and copying are grave and serious offences in the university and accept the penalties that would be imposed should we engage in plagiarism, collusion or copying. This assignment, or any part of it, has not been previously submitted by us or any other person for assessment on this or any other course of study.

Repository

The complete code and this report are available at: https://github.com/saskarov/garbage-recognizer